

```
GHCi, version 7.6.3: http://www.haskell.org/ghc/ :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
```

```
ghci> :m Database.Redis
```

```
ghci> conn <- connect defaultConnectInfo
```

```
Loading package array-0.4.0.1 ... linking ... done.
Loading package base-unicode-symbols-0.2.2.4 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package old-locale-1.0.0.5 ... linking ... done.
Loading package time-1.4.0.1 ... linking ... done.
Loading package transformers-0.3.0.0 ... linking ... done.
Loading package bytestring-0.10.0.2 ... linking ... done.
Loading package text-0.11.3.1 ... linking ... done.
Loading package stm-2.4.2 ... linking ... done.
Loading package primitive-0.5.0.1 ... linking ... done.
Loading package vector-0.10.0.1 ... linking ... done.
Loading package hashable-1.1.2.5 ... linking ... done.
Loading package transformers-base-0.4.1 ... linking ... done.
Loading package monad-control-0.3.2.1 ... linking ... done.
Loading package containers-0.5.0.0 ... linking ... done.
Loading package attoparsec-0.10.4.0 ... linking ... done.
Loading package mtl-2.1.2 ... linking ... done.
Loading package BoundedChan-1.0.3.0 ... linking ... done.
Loading package bytestring-lexing-0.4.3.2 ... linking ... done.
Loading package unix-2.6.0.1 ... linking ... done.
Loading package network-2.6.0.2 ... linking ... done.
Loading package resource-pool-0.2.3.2 ... linking ... done.
Loading package hedis-0.6.5 ... linking ... done.
```

```
ghci> runRedis conn ping
Right Ping
```

I would recommend that you use *defaultConnectInfo* to connect to the database, and its type is *ConnectInfo*:

```
ghci> :t defaultConnectInfo
defaultConnectInfo :: ConnectInfo
```

The different options that can be used in *defaultConnectInfo* are as follows:

```
connectHost      = "localhost"
connectPort      = PortNumber 6379 -- Redis port
connectAuth      = Nothing         -- No authentication
connectDatabase  = 0               -- SELECT database 0
connectMaxConnections = 10         -- Up to 10 connections
connectMaxIdleTime = 20            -- Keep connection open
                                   for 20 seconds
```

The types of *conn*, *connect*, *runRedis* and *ping* are given below:

```
ghci> :t conn
conn :: Connection
```

```
ghci> :t connect
connect :: ConnectInfo -> IO Connection
```

```
ghci> :t runRedis
runRedis :: Connection -> Redis a -> IO a
```

```
ghci> :t ping
ping :: RedisCtx m f => m (f Status)
```

If the Redis server was not started, and you tried to issue the *ping* command, the following exception would be automatically thrown by the package:

```
ghci> runRedis conn ping
*** Exception: connect: does not exist (No route to host)
```

You can automate the above code snippets into Haskell code with a main function, as demonstrated below:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.Redis
main :: IO (Either Reply Status)
main = do
    conn <- connect defaultConnectInfo
    runRedis conn ping
```

The *OverloadedStrings* extension allows string literals to be polymorphic for the *IsString* class. You can compile and run the above code inside GHCi, as follows:

```
$ ghci ping.hs
```

```
GHCi, version 7.6.3: http://www.haskell.org/ghc/ :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main                ( ping.hs, interpreted )
Ok, modules loaded: Main.
```

```
ghci> main
...
Right Ping
```

The *echo* Redis command is used to print a message that is passed as an argument to it. The equivalent *hedis echo* command expects the message to be of type *ByteString*. For example:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.Redis
import qualified Data.ByteString as B
```